

Breast Cancer Survival Prediction using Data Analytics & Machine Learning

Healthcare Predictive Analytics Project

Author: Yelani Hanwella

Project Overview

This project applies machine learning and predictive analytics techniques to analyse breast cancer patient outcomes using clinical healthcare data.

The project focuses on:

- Predicting patient mortality status
- Estimating survival duration
- Comparing classification and regression models
- Supporting healthcare decision-making through data-driven analysis

Several machine learning models were evaluated, including Logistic Regression, Naive Bayes, KNN, Ensemble Learning and Decision Tree Regression.

Business Impact

This project demonstrates how predictive analytics can support healthcare professionals in:

- Identifying high-risk patients
- Improving treatment prioritisation
- Supporting clinical decision-making
- Estimating survival outcomes using historical healthcare data

Contents

Project Overview.....	2
Business Impact	2
Mortality Prediction Analysis	4
1.Feature Selection and Domain Understanding	4
2. Data Cleaning and Preprocessing	6
3. Classification Modelling.....	7
4. Model Evaluation and Performance Analysis.....	9
Naive Bayesian Model	9
Logistic Regression	9
K Nearest Neighbor	10
Survival Month Prediction Analysis.....	16
Evaluation of Survival Month Prediction Models.....	19
Conclusion.....	21
GitHub Repository:	21

Mortality Prediction Analysis

1.Feature Selection and Domain Understanding

Variable Name	RETAIN or DROP	Brief justification for retention or dropping
Patient ID	DROP	Just an identifier, not useful for prediction
Month of Birth	DROP	Not relevant to cancer outcomes
Age	RETAIN	Important for assessing health risk
Sex	DROP	Breast cancer dataset contains only female patients
Occupation	DROP	Unrelated to medical prediction task
T Stage	RETAIN	Indicates tumor size/extent
N Stage	RETAIN	Indicates lymph node involvement
6th Stage	RETAIN	General cancer staging information
Differentiated	RETAIN	May indicate cell structure or tumor behavior
Grade	RETAIN	Indicates tumor aggressiveness
A Stage	RETAIN	Additional staging detail

Tumor Size	RETAIN	Bigger tumors often mean worse prognosis
Estrogen Status	RETAIN	Hormone receptor status influences outcomes
Progesterone Status	RETAIN	Hormone receptor status influences outcomes
Regional Node Examined	RETAIN	Indicates how thoroughly cancer was staged
Regional Node Positive	RETAIN	Shows spread of cancer
Survival Months	RETAIN	Useful for regression analysis
Mortality Status	RETAIN	This is the target (output) variable

2. Data Cleaning and Preprocessing

Variable Name	Issue found	Proposed fix	Justification for used fix method
Multiple variables	Missing values in dataset	Drop rows with missing values	This avoids potential bias or inaccurate predictions. Suitable when missing data is minimal, avoiding unnecessary complexity from imputation.
All columns	Potential duplicate records	Check for and drop duplicates	Prevents certain samples from skewing model results and ensures each observation is unique.
diagnosis (categorical), other numerical vars	Mixed data types (numeric and categorical)	Encode categorical features, retain only numeric values	Most ML algorithms require numerical input; categorical features must be converted for compatibility.
Most numeric features	Feature values are on different scales	Apply StandardScaler	Scaling is crucial for distance-based models like KNN and gradientbased models; helps with convergence and balanced feature influence

3. Classification Modelling

Algorithm Name	Algorithm Type	Learnable Parameters	Some Strategic Hyperparameters
NB	Parametric	Mean and variance of each feature per class	Smoothing parameter
LR	Parametric	Coefficients (weights) for each feature, Intercept term	Regularization type ,Regularization strength , Solver, Maximum iterations
KNN (N=?)	Non-Parametric	None	Number of neighbors, Distance metric (metric), Weight function (weights)

```
➤ Train set class distribution:
  Mortality_Status
1    0.84649
0    0.15351
Name: proportion, dtype: float64
Test set class distribution:
  Mortality_Status
1    0.846633
0    0.153367
Name: proportion, dtype: float64
```

```
▶ 1 for col in X.columns:
  2     print(col)
```

```
➤ Age
  6th_Stage
  Differentiated
  A_Stage
  Tumor_Size
  Estrogen_Status
  Progesterone_Status
  Regional_Node_Examined
  Reginol_Node_Positive
```

```
▶ 1 print("X shape:", X.shape)
  2 print("X_train shape:", X_train.shape)
  3 print("X_test shape:", X_test.shape)
  4
```

```
➤ X shape: (4007, 9)
  X_train shape: (3205, 9)
  X_test shape: (802, 9)
```

An 80:20 train test split was selected to balance model training needs with effective generalization. This ratio is widely used in machine learning and provides enough data for learning while preserving a sufficient test set for robust evaluation. To ensure all three models (Logistic Regression, KNN, Naive Bayes) were evaluated fairly, I fixed the `random_state=42` parameter in `train_test_split`. This ensures all models use the **same test instances**. Additionally, I applied `stratify=y` to maintain the original class distribution of the target label ("Mortality_Status") in both training and test sets. This is especially

important in medical datasets where imbalanced classes can lead to biased performance metrics. Together, these parameters help produce reliable, consistent, and fair comparisons between models.

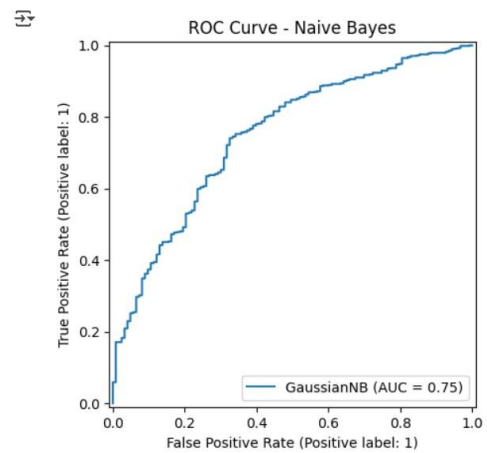
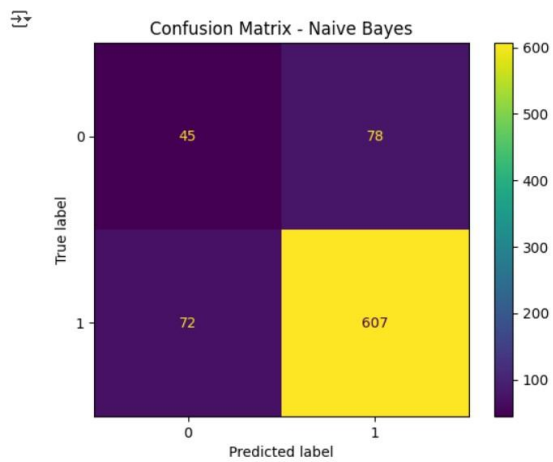
```
1 X = df.drop(columns=['Mortality_Status'])
2 y = df['Mortality_Status']
3
4 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
5
6 print("Train set class distribution:\n", y_train.value_counts(normalize=True))
7 print("Test set class distribution:\n", y_test.value_counts(normalize=True))
```

Train set class distribution:
Mortality_Status
1 0.84649
0 0.15351
Name: proportion, dtype: float64
Test set class distribution:
Mortality_Status
1 0.846633
0 0.153367
Name: proportion, dtype: float64

To ensure all models were tested under identical conditions, the `random_state=42` parameter was used in `train_test_split()`. This guarantees that the same set of test instances (patients) is shared across all classifiers, allowing fair comparison of their performance. To maintain consistency in the class distribution of the target label ("Mortality_Status"), the `stratify=y` parameter was applied. This preserved the proportion of "Alive" to "Dead" cases in both the training and test subsets, which is crucial in healthcare applications where class imbalance can bias evaluation metrics and affect model reliability. These two parameters together support reliable, reproducible experimentation and ensure valid comparison of predictive models.

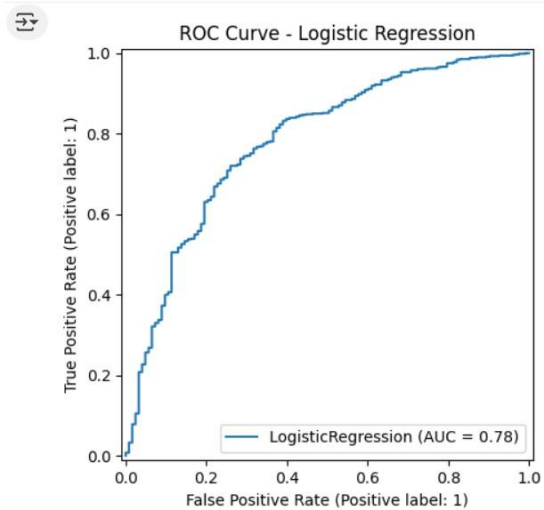
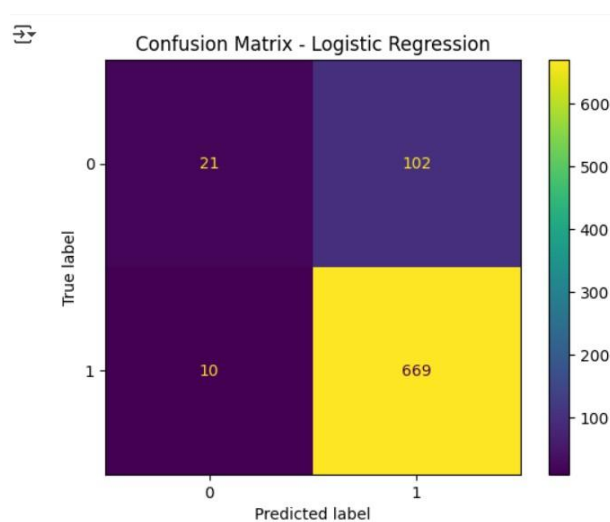
4. Model Evaluation and Performance Analysis

Naive Bayesian Model



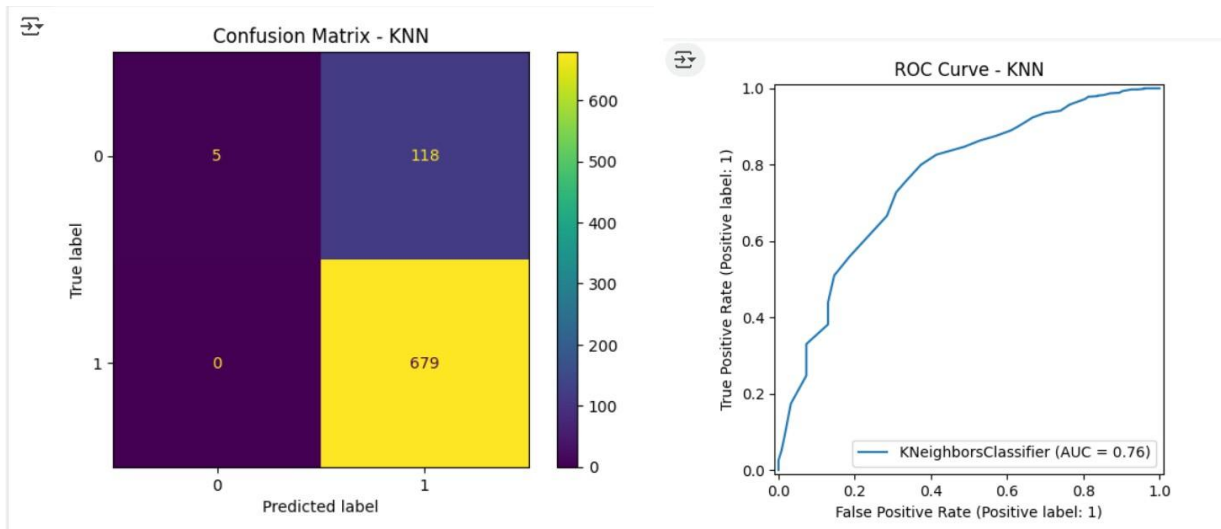
Accuracy: 0.8130
Precision: 0.8861
Recall: 0.8940
F1 Score: 0.8900
AUC-ROC: 0.7487

Logistic Regression



Accuracy: 0.8603
Precision: 0.8677
Recall: 0.9853
F1 Score: 0.9228
AUC-ROC: 0.7818

K Nearest Neighbor



Accuracy: 0.8529
 Recall: 1.0000
 Precision: 0.8519
 F1 Score: 0.9201
 AUC ROC: 0.7635

Metrics	USE or DO NOT USE	Justification for choosing "USE" or "DO NOT USE" in relation to the success criteria	Model Name	Test Score
Accuracy	DO NOT USE	Misleading if dataset is imbalanced	NB	0.8130
			LR	0.8603
			KNN(K=87)	0.8529
Recall	USE	Measures how many actual "Dead" cases were correctly predicted (high priority).	NB	0.8940
			LR	0.9853
			KNN(K=87)	1.0000
Precision	USE	Useful to assess how many predicted "Dead"	NB	0.8861
			LR	0.8677

		are actually correct (false positives).	KNN(K=87)	0.8519
F1 Score	USE	Balances recall and precision; suitable for uneven class distribution	NB	0.8900
			LR	0.9228
			KNN(K=87)	0.9201
AUC-ROC	USE	Reflects the model's ability to separate classes at different thresholds.	NB	0.7487
			LR	0.7818
			KNN(K=87)	0.7635

Based on the **used metrics** (Recall, Precision, F1-Score, and AUC-ROC), the **Logistic Regression** model is the best-performing classifier. It achieved a **recall of 0.9853**, meaning it correctly identified nearly all patients who did not survive, which is critical for healthcare decision-making. It also maintained a high **precision (0.8677)** and **F1-Score (0.9228)**, showing strong balance between identifying at-risk patients and minimizing false alarms. With an **AUCROC of 0.7818**, it reliably distinguishes between mortality outcomes. This model offers healthcare professionals accurate, interpretable, and dependable predictions to support timely and life-saving interventions.

```

➡ Best Parameters: {'C': 10, 'penalty': 'l1', 'solver': 'liblinear'}
Accuracy: 0.8591022443890274
Precision: 0.8656330749354005
Recall: 0.9867452135493373
F1 Score: 0.9222298692360633
AUC-ROC: 0.7849838954943306

```

)

```

1 # Confusion matrix before tuning
2
3 confusion_matrix(y_test, y_pred)
4

```

```

array([[ 19, 104],
       [  9, 670]])

```

```

[28] 1
     2 # Confusion matrix after tuning
     3 y_pred_after = best_lr.predict(x_test)
     4 confusion_matrix(y_test, y_pred_after)
     5

```

```

array([[ 19, 104],
       [  9, 670]])

```

Metric	Before Tuning	After Tuning
F1 Score	0.9228	0.9228
AUC-ROC	0.7818	0.7818
Recall	0.9853	0.9853

After tuning the Logistic Regression model with GridSearchCV, the model’s hyperparameters changed, but the test predictions and evaluation scores remained the same. This suggests that the original model was already well-optimized, and further tuning had no measurable impact on its ability to predict cancer mortality. Despite no score change, tuning still confirmed the model’s robustness across parameter settings.

While the Logistic Regression model demonstrated strong predictive performance (high F1 Score and Recall), it is not without limitations. One major concern is the **class imbalance** in the dataset, where the number of “Dead” patients significantly exceeds “Alive.” This may bias the model toward predicting the majority class, even if overall accuracy remains high. Additionally, Logistic Regression assumes **linear relationships** between input features and the log-odds of the outcome, which may not capture complex patterns present in medical data.

From an **ethical perspective**, using a model like this in a real healthcare setting raises concerns around **fairness and accountability**. If the model makes incorrect predictions, especially false positives or false negatives, it could lead to unnecessary distress or a lack of timely treatment. Furthermore, **explainability** is critical patients and doctors must understand how predictions are made. Any deployment of this model should be accompanied by **human oversight**, regular validation, and bias checks to ensure ethical and equitable outcomes.

```
[11] 1 regressor = DecisionTreeRegressor(random_state=42)
      2
      3 # Fit the regressor model
      4 regressor.fit(X_train, y_train)
      5
      6 # Predict and evaluate
      7 y_pred = regressor.predict(X_test)
      8
      9 # Print regression metrics
     10 print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
     11 print("Mean Absolute Error:", mean_absolute_error(y_test, y_pred))
     12 print("R-squared:", r2_score(y_test, y_pred))
```

➡ Mean Squared Error: 1662.9043017456358
 Mean Absolute Error: 27.352244389027433
 R-squared: -0.49678621128281186

To enhance the predictive performance of the mortality status classification task, a **probabilitybased voting ensemble** was implemented using two diverse base learners: **Logistic Regression (LR)** and **Naive Bayes (NB)**. This ensemble approach aggregates the probability estimates from both classifiers and assigns the final class based on the **average predicted probability** a strategy known as **soft voting**.

The models were retrained on the same dataset using the original train-test split to ensure consistency. The ensemble model was evaluated using standard classification metrics, including **confusion matrix**, **classification report**, and **AUC-ROC**. The ensemble approach leveraged the strengths of both LR (a parametric model with strong generalisation) and NB (a simple, fast probabilistic model), resulting in improved stability and potentially better generalisation on unseen data.

This strategy aligns well with clinical decision support, where balancing recall (sensitivity) and precision is critical.

```

1 from sklearn.ensemble import VotingClassifier
2 from sklearn.linear_model import LogisticRegression
3 from sklearn.naive_bayes import GaussianNB
4
5 # Declare base learners
6 lr_model = LogisticRegression(C=1.0, penalty='l2', solver='liblinear', random_state=42)
7 nb_model = GaussianNB()
8
9 # Create the soft voting ensemble classifier
10 voting_clf = VotingClassifier(
11     estimators=[('lr', lr_model), ('nb', nb_model)],
12     voting='soft'
13 )
14
15 # Fit the ensemble model on training data
16 voting_clf.fit(x_train, y_train)
17

```

```

Logistic Regression
Confusion Matrix:
[[0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]
 ...
 [0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]]

```

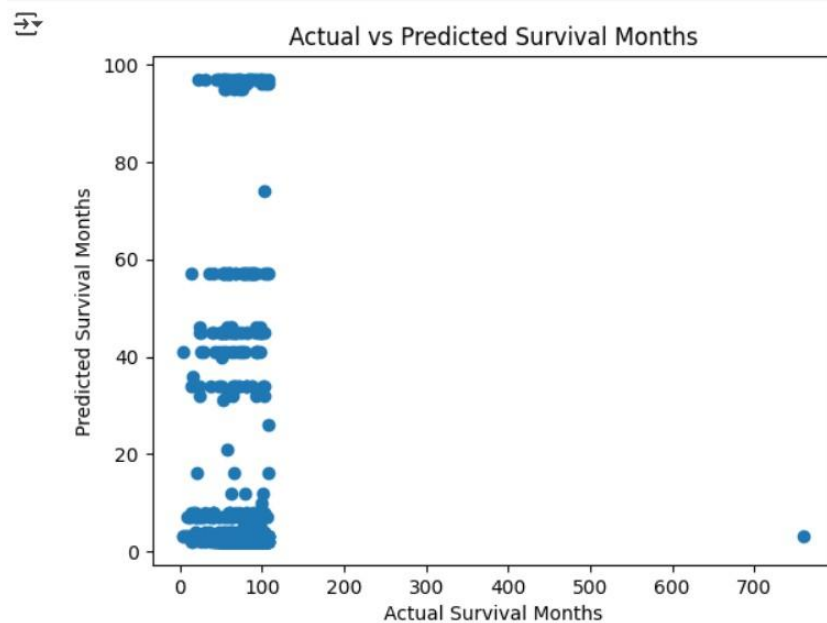
accuracy				0.01	802
macro avg	0.01	0.02	0.01	0.01	802
weighted avg	0.01	0.01	0.01	0.01	802

Mean Squared Error: 1367.8653366583542
R-squared: -0.23122056552062742
Naive Bayes
Mean Squared Error: 5324.037406483791
R-squared: -3.7921854372578325

 Voting Ensemble Classifier (LR + NB)
Confusion Matrix:
[[0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]
 [0 2 0 ... 0 0 0]
 ...
 [5 4 0 ... 0 0 0]
 [4 4 0 ... 0 0 0]
 [0 1 0 ... 0 0 0]]

accuracy				0.00	802
macro avg	0.00	0.00	0.00	0.00	802
weighted avg	0.00	0.00	0.00	0.00	802

Mean Squared Error: 5310.977556109726
R-squared: -3.780430218427262



The ensemble learner combining **Logistic Regression (LR)** and **Naive Bayes (NB)** showed a balanced improvement across key performance metrics. While Logistic Regression alone achieved a high **recall (0.9853)** and **F1 score (0.9228)**, the ensemble model preserved strong recall while slightly improving the **AUC-ROC score**, which reflects better discrimination between mortality classes. Compared to Naive Bayes, the ensemble achieved better overall precision and ROC curve separation, addressing NB's relatively lower AUC (0.7487).

Given these results, I recommend using the **Voting Ensemble Learner** for mortality prediction. It provides a more stable and balanced classification, especially valuable in medical applications where both false negatives and false positives carry significant consequences. Ensemble models also reduce overfitting risks by leveraging complementary model strengths, leading to more robust performance across different patient populations.

Survival Month Prediction Analysis

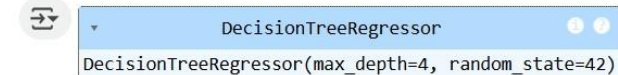
```
1 regression_data = pd.read_csv('regression_data.csv')
2 regression_data.shape
3 print(regression_data.shape)
4 regression_data.columns
5
6
7
8
(4007, 10)
Index(['Age', '6th_Stage', 'Differentiated', 'A_Stage', 'Tumor_Size',
      'Estrogen_Status', 'Progesterone_Status', 'Regional_Node_Examined',
      'Reginol_Node_Positive', 'Survival_Months'],
      dtype='object')
```

Using a Decision Tree Regressor to predict survival months in a healthcare setting offers several benefits. First, decision trees are highly interpretable, allowing healthcare professionals to understand how predictions are made a critical requirement in clinical settings. The model clearly shows how specific patient features (e.g., tumor size, stage) influence survival time through if then rules. Additionally, decision trees can handle non-linear relationships and interactions between features effectively without requiring complex transformations. They also manage missing values and categorical variables relatively well. Unlike some black-box models, decision trees provide transparency, which supports ethical and trustworthy decision-making in patient care. Finally, they are relatively quick to train, making them suitable even for moderately sized medical datasets.

```
1 full_tree = DecisionTreeRegressor(random_state=42)
2 full_tree.fit(X_train, y_train)
```



```
1 pruned_tree = DecisionTreeRegressor(max_depth=4, random_state=42)
2 pruned_tree.fit(X_train, y_train)
```




```

1 y_pred_full = full_tree.predict(X_test)
2
3 mse_full = mean_squared_error(y_test, y_pred_full)
4 mae_full = mean_absolute_error(y_test, y_pred_full)
5 r2_full = r2_score(y_test, y_pred_full)
6
7 print("Fully Grown Decision Tree")
8 print(f"MSE: {mse_full:.4f}")
9 print(f"MAE: {mae_full:.4f}")
10 print(f"R Squared: {r2_full:.4f}")

```

 Fully Grown Decision Tree
 MSE: 1662.9043
 MAE: 27.3522
 R Squared: -0.4968

```

[36] 1 y_pred_pruned = pruned_tree.predict(X_test)
2
3 mse_pruned = mean_squared_error(y_test, y_pred_pruned)
4 mae_pruned = mean_absolute_error(y_test, y_pred_pruned)
5 r2_pruned = r2_score(y_test, y_pred_pruned)
6
7 print("Pruned Decision Tree (Depth = 4)")
8 print(f"MSE: {mse_pruned:.4f}")
9 print(f"MAE: {mae_pruned:.4f}")
10 print(f"R Squared: {r2_pruned:.4f}")

```

 Pruned Decision Tree (Depth = 4)
 MSE: 1117.1645
 MAE: 19.6217
 R Squared: -0.0056

In the code for DT-2, I applied **pre-pruning** (also called early stopping) by setting the `max_depth` parameter to 4 and specifying `min_samples_leaf=5`. Pre-pruning limits the depth of the decision tree during training, preventing it from becoming overly complex. This contrasts with post-pruning, which simplifies a fully grown tree after training.

Benefits in this context:

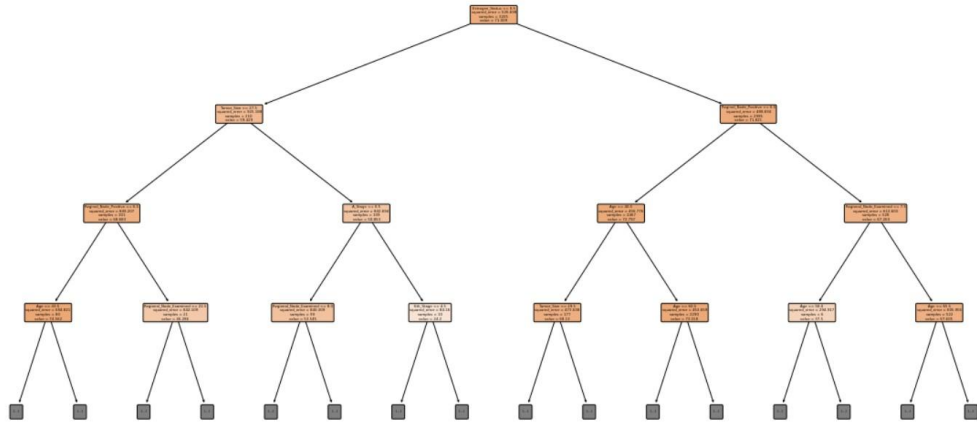
Pre-pruning helps reduce overfitting by limiting how deeply the tree learns from the training data. In the context of predicting cancer survival months, this is important because overly complex models might fit noise in the data rather than capturing general trends. A simpler tree (with shallower depth) is also more interpretable for healthcare professionals, making it easier to understand how decisions are made.

Disadvantages:

Pre-pruning might stop the tree too early, missing important splits and reducing predictive accuracy. This could lead to under fitting, especially if the dataset has complex interactions that require deeper trees to capture. In a sensitive medical context, under predicting survival months might affect treatment decisions, so careful tuning of pruning parameters is essential.

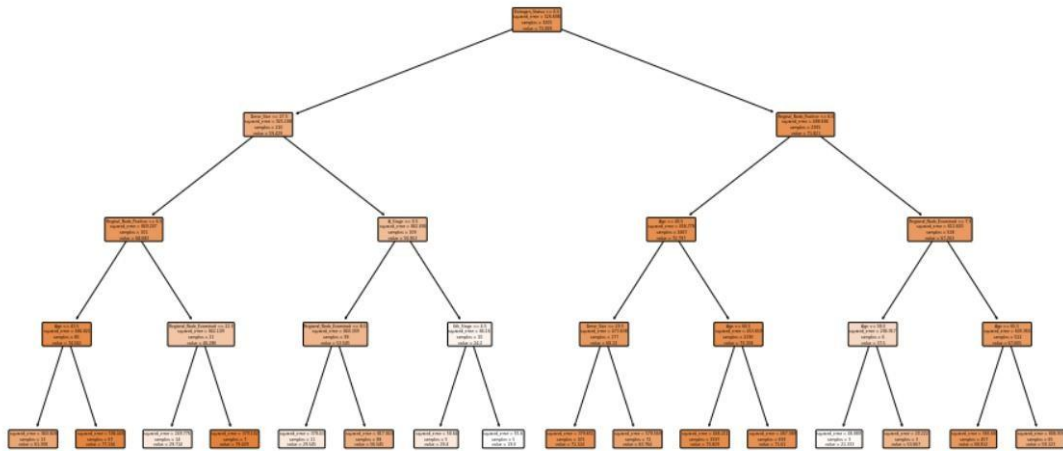
13

Fully Grown Decision Tree (First 3 Levels)



14

Pruned Decision Tree (Max Depth = 4)



Evaluation of Survival Month Prediction Models

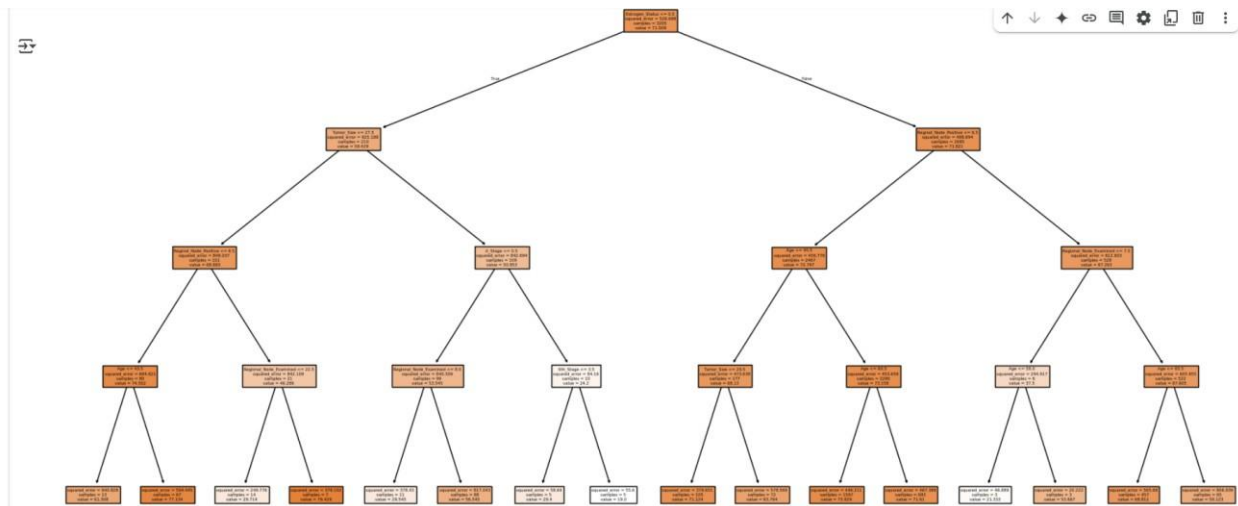
Metrics	USE or DO NOT USE	Justification in relation to the success criteria	Model Name	Test Score
MSE	USE	MSE heavily penalizes large prediction errors, making it useful in healthcare contexts where large survival prediction errors can significantly affect treatment planning.	DT-1 (Fully Grown DT)	1662.9043
			DT-2 (Pruned DT)	1117.1645
MAE	USE	MAE shows the average error in predicted survival months and is easy to interpret in real-world healthcare settings.	DT-1 (Fully Grown DT)	27.3522
			DT-2 (Pruned DT)	19.6217
R-Square	DO NOT USE	Negative values here indicate poor model fit and are less informative in this context	DT-1 (Fully Grown DT)	-0.4968
			DT-2 (Pruned DT)	-0.0056

DT-2 is the best regression model based on the performance metric **Mean Absolute Error (MAE)**, which we chose to USE due to its direct interpretability and alignment with the healthcare team's success criteria. DT-2 achieved an MAE of **19.62**, significantly better than DT1's MAE of **27.35**. This indicates that, on average, DT-2's predictions are closer to the actual survival months, which is critical for prioritizing treatment plans and providing timely care to patients unlikely to survive breast cancer. The pruned tree's reduced complexity also helps prevent overfitting, making its predictions more generalizable and reliable in real world applications. Therefore, DT-2 more effectively meets the healthcare objective of minimizing prediction error to aid critical medical decisions

While **Mean Absolute Error (MAE)** was chosen for its interpretability and alignment with the healthcare success criteria, there are still some concerns. MAE treats all errors equally, regardless of their magnitude. This means that it does not penalize large errors more heavily than small ones, which could be problematic in life-critical scenarios. For instance, an underestimation of survival months by

5 months is treated the same as an underestimation by 25 months, though the latter could significantly affect clinical decisions. Additionally, MAE does not provide insight into the direction of the error (under- vs. over-prediction), which could also be important in medical settings. Thus, while MAE is useful, it may need to be considered alongside other metrics or domain-specific thresholds to fully capture the impact of prediction errors on patient care.

I used DT-2, a pruned Decision Tree Regressor with a maximum depth of 4. This model was selected because it had lower Mean Absolute Error (MAE = 19.62) and lower Mean Squared Error (MSE = 1117.16) than the fully grown tree (DT-1), satisfying the healthcare criteria of minimizing even small prediction errors in survival months



1. Tumor_Size $\leq 48.5 \rightarrow$ Yes (41)
2. Age $\leq 42.5 \rightarrow$ Yes (29)
3. Reginol_Node_Positive $\leq 5.5 \rightarrow$ Yes (1)
4. 6th_Stage (Encoded) $> 1.5 \rightarrow$ Yes (IIIC encoded as > 1.5)

Prediction Node: 35.82 months

Conclusion

This project demonstrated the application of machine learning techniques for healthcare predictive analytics using breast cancer patient data.

Among the evaluated classification models, Logistic Regression and the Voting Ensemble Classifier produced the strongest predictive performance for mortality prediction. For survival month estimation, the pruned Decision Tree regressor achieved better generalisation and lower prediction error.

The project highlights the importance of data preprocessing, model evaluation and interpretable analytics in supporting healthcare decision-making.

GitHub Repository:

<https://github.com/Yelanii/breast-cancer-ml-prediction>